University of Southern Denmark

Department of Mathematics and Computer Science

SPD801: Master Thesis in Computer Science

Formalising πLL in Lean

Author

Mikkel Brix Nielsen
mikke21

Supervisors

Supervisor: Fabrizio Montesi
Cosupervisor: Marco Peressotti
Cosupervisor: Xueying Qin

12th May 2026

1 Introduction

The Curry–Howard correspondence establishes a deep connection between logic and computation, where proofs correspond to programs and logical propositions correspond to types. This perspective has been particularly influential in the study of concurrent systems, where linear logic and session types provide a logical foundation for structured communication.

In this setting, linear logic captures resource-sensitive reasoning, while session types describe communication protocols between concurrent processes. The resulting correspondence, often referred to as propositions-as-sessions, provides a principled interpretation of interaction as logical proof transformation, and has been further related to process calculi such as the π-calculus.

This line of work has been further developed by [?], who provide a unified account reconciling linear logic, the π-calculus, and their metatheory. Their framework serves as the formal basis for the system πLL considered in this thesis.

Motivation and correctness concerns. Despite the elegance of these theoretical correspondences, their informal and paper-based presentations are prone to subtle errors. In particular, the application of inference rules in linear logic and session-typed systems often relies on implicit ordering conventions and intuitive interpretations of syntax. This can lead to mistakes that are not immediately apparent in written derivations.

As an illustrative example, small inconsistencies can arise in the presentation of typing rules within such systems, where the intuitive alignment between variable ordering and type assignment does not precisely match the formal rule structure. These discrepancies are not flaws of the underlying theory, but rather reflect the difficulty of maintaining precision in informal presentations, including in the framework of Montesi and Peressotti.

As observed in discussions with the authors of the framework, such issues are easy to overlook because rule application is often guided by intuition rather than strict syntactic structure. This highlights that even formally correct rules may be unintuitive to apply due to their representation.

These observations motivate a more disciplined approach to formal reasoning, where correctness is ensured by construction rather than by inspection.

Why mechanization. Mechanization provides a means of bridging this gap. Rather than extending the underlying theory, the goal of mechanization in this work is to validate and internalize existing results within a proof assistant. This allows us to eliminate ambiguity arising from informal presentation and to ensure that all derivations are checked with respect to a precise formal semantics.

Moreover, mechanized developments have repeatedly demonstrated their ability to uncover small but meaningful inconsistencies in paper proofs, supporting the view that proof assistants serve as a valuable tool for verifying, rather than merely implementing, theoretical frameworks.

Contributions. In this thesis, we present a mechanized formalization of the π-calculus-based linear logic system πLL as developed by Montesi and Peressotti, with particular emphasis on its multiplicative fragment. The contributions are as follows:

- A formalization of the syntax and typing system of πLL in Lean, including processes, session types, and substitution operations.
- A formalization of the operational semantics for the multiplicative fragment of πLL.
- Formal proofs of session fidelity for the multiplicative fragment.
- A locally nameless representation of binding, eliminating the need for reasoning up to α-equivalence and simplifying substitution reasoning.

Scope of the formalization. This thesis focuses on a mechanized reconstruction of the πLL system of [?], with particular emphasis on its multiplicative fragment. The syntactic and typing components

of πLL have been fully mechanized, including processes, session types, typing judgments, and substitution operations. The operational semantics are defined for the multiplicative fragment, with the exception of the res-rule.

Consequently, metatheoretic results concerning typing apply to the full system, while results involving operational semantics are restricted to the multiplicative fragment. The aim of this work is not to extend the theory, but to provide a mechanically verified account of an existing theoretical framework.

2 Background

This section begins by introducing the theoretical foundations underlying the formalisation following the presentation of [Montesi and Peressotti 2021], We cover the full πLL calculus, including processes, types, typing, environments / hyperenvironments, and judgements, then restrict attention to the multiplicative fragment πMLL when discussing operational semantics.

Following [Montesi and Peressotti 2021], we use *blue* to highlight types and *red* for process terms throughout this thesis.

2.1 Classical Linear Logic, CLL

Linear logic [Girard 1987] is a refinement of classical and intuitionistic logic. In contrast to these systems, where formulas can be freely duplicated or discarded, linear logic restricts structural rules such as weakening and contraction, and instead treats formulas as consumable resources. This yields a resource-sensitive interpretation of logic, where each assumption must be used in a controlled manner [Di Cosmo and Miller 2023]. In this thesis, we work in the classical presentation of linear logic, where sequents are symmetric and duality replaces the distinction between assumptions and conclusions.

In particular, conjunction and disjunction split into multiplicative and additive variants. The multiplicative fragment forms the core of the system and includes the tensor connective $A \otimes B$ and its dual $A \parr B$, together with the corresponding units 1 and $\bot$. These connectives capture composition and interaction of resources.

Intuitively, $A \otimes B$ describes the joint availability of two independent resources, while $A \parr B$ captures their dual form of interaction. Under a process interpretation, $\otimes$ can be read as producing a value of type A and continuing as B, whereas $A \parr B$ corresponds to receiving a value of type A and proceeding as B. The units 1 and $\bot$ represent the absence of further output and input, respectively. A simplified presentation of these rules is given below:

$$\frac{}{\vdash 1}\,1 \qquad \frac{\vdash \Gamma}{\vdash \Gamma, \bot}\,\bot \qquad \frac{\vdash \Gamma, A \quad \vdash \Delta, B}{\vdash \Gamma, \Delta, A \otimes B}\,\otimes \qquad \frac{\vdash \Gamma, A, B}{\vdash \Gamma, A \parr B}\,\parr$$

Here rule $\otimes$ presents a simplified form of the tensor rule from classical linear logic. In general, the tensor connective $A \otimes B$ combines two independent derivations. It requires a proof of A and a proof of B, each obtained from disjoint contexts, and produces a proof of the composite resource $A \otimes B$. Intuitively, this corresponds to combining two independent resources into a single composite resource, and can be read as producing a value of type A and then continuing as B.

The unit rule rule 1 introduces the unit 1, which represents the absence of further obligations. It can be understood as an "empty output", corresponding to a computation that performs no further interaction, and serves as the neutral element of tensor.

Dually, rules $\bot$ and $\parr$ describe the behavior of the connective $A \parr B$ and its unit $\bot$. The $\parr$ rule combines resources within a single context, reflecting a form of interaction where both components

are made available to the surrounding context. From an operational perspective, this corresponds to interacting with an input of type A while continuing as B. The unit $\bot$ represents the absence of further input, acting as an "empty input" and serving as the dual neutral element.

For example, two independent instances of the unit 1 can be introduced by the rule 1 and combined via rule $\otimes$ to derive $1 \otimes 1$:

$$\dfrac{\dfrac{}{\vdash 1}\,1 \quad \dfrac{}{\vdash 1}\,1}{\vdash 1 \otimes 1}\otimes$$

Here each leaf introduces one resource independently, and the tensor rule combines them into a single compound resource $1 \otimes 1$. After this step, the individual resources are no longer available separately, reflecting the resource-sensitive nature of the system.

This resource-oriented view of logic provides the foundation for its applications in computer science, particularly in the study of concurrency and session-typed communication. In the following sections, this perspective is used to motivate the process interpretation underlying πLL.

2.2 The π-Calculus: Processes and Communication

The π-calculus [Milner et al. 1992] is a process calculus for modeling concurrent computation with dynamic communication topology, where channel names themselves can be transmitted, allowing the communication network to evolve during execution. We follow the presentation of Montesi and Peressotti [2021], which adopts Wadler's convention of using square brackets '[-]' for output and round parentheses '(-)' for input.

Programs in πMLL are processes (P, Q, R, ...), which communicate over names (x, y, z, ...) representing session endpoints. Sessions are binary (they involve exactly two endpoints), a discipline enforced by the typing system introduced in 2.3. We assume an infinite set of names with decidable equality. The process terms of πMLL are defined by the following grammar:

$P, Q ::=$	$x[y].P$	*output y on x and continue as P*
$\mid$	$x(y).P$	*input y on x and continue as P*
$\mid$	$x[].P$	*output (empty message) on x and continue as P*
$\mid$	$x().P$	*input (empty message) on x and continue as P*
$\mid$	$\nu xy\,P$	*name restriction (cut)*
$\mid$	$P \mid Q$	*parallel composition*
$\mid$	$x[\mathrm{L}].P$	*select left on x and continue as P*
$\mid$	$x[\mathrm{R}].P$	*select right on x and continue as P*
$\mid$	$x.\mathrm{case}\{\mathrm{L}:P, \mathrm{R}:Q\}$	*offer a binary choice between P or Q on x*
$\mid$	$x[A].P$	*output type A on x and continue as P*
$\mid$	$x(X).P$	*input a type as X on x and continue as P*
$\mid$	$!x\langle z_1, \ldots, z_n\rangle.\{P\}$	*server (replicable process) on x depending on servers on $z_1 \ldots z_n$*
$\mid$	$x[\mathrm{USE}].P$	*consume a server on x and continue as P*
$\mid$	$x[\mathrm{DUP}](x').P$	*duplicate server x as x' in P*
$\mid$	$x[\mathrm{DISP}].P$	*dispose of a server on x*
$\mid$	$x \leftrightarrow y$	*link x and y*
$\mid$	$\mathbf{0}$	*terminated process*

Term $x[y].P$ allocates a fresh name y, outputs it over x, and proceeds as P. Dually, $x(y).P$ inputs a name over x, binding it to y in P. Terms $x[].P$ and $x().P$ model empty output and input.

Restriction $\nu xy\,P$ connects the two endpoints x and y of P to form a session, where the order of x and y is irrelevant and $\nu xy\,P = \nu yx\,P$. Parallel composition $P \mid Q$ runs P and Q concurrently, and $\mathbf{0}$ is the terminated process, acting as the unit of parallel composition.

Example 2.1. Consider the process $P \triangleq \nu xz\,(x[].\mathbf{0} \mid z().y[].\mathbf{0})$. This process creates a private session between the restricted endpoints x and z. The left process, $x[].\mathbf{0}$, signals termination on x before terminating, while the right process, $z().y[].\mathbf{0}$, waits for a termination signal on z before proceeding by sending a termination signal on y and then terminating.

This calculus provides the operational foundation for πLL. In the following section, we introduce the type system that governs how names are used, ensuring that each session endpoint is used exactly according to its protocol.

2.3 Types and Propositions-as-Sessions Correspondence

In πLL, session types are linear logic propositions [Caires and Pfenning 2010; Montesi and Peressotti 2021; Wadler 2014]. Each type describes the communication protocol of a channel endpoint, and duality, the key structural feature of classical linear logic, ensures that the two endpoints of every session implement complementary protocols.

The correspondence between linear logic and session types as presented in [Montesi and Peressotti 2021] following [Carbone et al. 2016; Wadler 2014] is captured directly by the type grammar. Each connective denotes a communication action, where $A \otimes B$ describes sending a value of type A and continuing as B, while its dual $A \parr B$ describes receiving. The units $\mathbf{1}$ and $\bot$, again, represent the absence of out and input, respectively. The additive connectives $A \oplus B$ and $A \,\&\, B$ capture choice in the sense of selection and offering of alternatives, while the exponentials $!A$ and $?A$ govern replication. The $\forall X.A$ and $\exists X.A$ allow polymorphic session behavior. The type grammar of πLL is defined as follows:

$A, B ::=$	$A \otimes B$	*send A, proceed as B*	$\mid$	$A \parr B$	*receive A, proceed as B*
$\mid$	$\mathbf{1}$	*empty output, unit for* $\otimes$	$\mid$	$\bot$	*empty input, unit for* $\parr$
$\mid$	$A \,\&\, B$	*offer A or B*	$\mid$	$A \oplus B$	*select A or B*
$\mid$	$\exists X.A$	*existential, type output*	$\mid$	$\forall X.A$	*universal, type input*
$\mid$	$?A$	*client request*	$\mid$	$!A$	*server accept*
$\mid$	X	*type variable*	$\mid$	$X^{\bot}$	*dual of type variable*

Duality is an involution, $(A^{\bot})^{\bot} = A$, defined structurally on types by swapping each connective with its dual. This ensures that for any well-typed session, the two endpoints always carry dual types, guaranteeing coherent interaction by construction:

$$(A \otimes B)^{\bot} = A^{\bot} \parr B^{\bot} \qquad \mathbf{1}^{\bot} = \bot \qquad (A \oplus B)^{\bot} = A^{\bot} \,\&\, B^{\bot}$$
$$(!A)^{\bot} = ?A^{\bot} \qquad (\exists X.A)^{\bot} = \forall X.A^{\bot} \qquad (X)^{\bot} = X^{\bot}$$

2.4 Environments, Hyperenvironments and Judgements

Typing in πLL is organised into two layers: environments track how individual names are typed, while hyperenvironments track which names are used independently in parallel.

Following the presentation in [Montesi and Peressotti 2021], *environments* ($\Gamma, \Delta, \Xi, \ldots$) are defined as lists allowing exchange, meaning order is irrelevant. They can be written as $\Gamma = x_1 : A, \ldots, x_n : A_n$, where each x_i is associated to its respective A_i, for $1 \leq i \leq n$. Environments can be merged as long as they do not share any names for example assume Γ and Δ do not share any names, then

Γ, Δ is the environment containing exactly the associations in Γ and Δ regardless of ordering. Environments act as a partial commutative monoid, using ',' as the sum and the empty environment $\bullet$ as unit:

$$\Gamma, \bullet = \Gamma \qquad \Gamma, \Delta = \Delta, \Gamma \qquad (\Gamma, \Delta), \Xi = \Gamma, (\Delta, \Xi)$$

Environments dictate how names are used, but it does not distinguish whether names are used independently or in parallel. That role is handled by handled by *hyperenvironments* ($\mathcal{G}, \mathcal{H}, \mathcal{I}, \ldots$), which are collections of environments joined using the parallel operator '$\mid$'. They can be written as follows $\mathcal{G} = \Gamma_1, \ldots, \Gamma_n$, where $\Gamma_1, \ldots, \Gamma_n$ is a collection of environments. Given $\mathcal{G}$ and $\mathcal{H}$ having no names in common then $\mathcal{G} \mid \mathcal{H}$ is the hyperenvironment containing exactly the environments of $\mathcal{G}$ and $\mathcal{H}$, ignoring order. Like environments, all names in a hyperenvironment are unique, they can be combined as long as they do not share any names, and they act as a partial commutative monoid using '$\mid$' the sum and $\varnothing$ as unit, where $\varnothing$ is the hyperenvironment containing no environments:

$$\mathcal{G} \mid \varnothing = \mathcal{G} \qquad \mathcal{G} \mid \mathcal{H} = \mathcal{H} \mid \mathcal{G} \qquad (\mathcal{G} \mid \mathcal{H}) \mid \mathcal{I} = \mathcal{G} \mid (\mathcal{H} \mid \mathcal{I})$$

Judgements, written as $\vdash P :: \mathcal{G}$, states that the process P uses its free names in accordance with $\mathcal{G}$ and can be derived using the inference rules displayed in Figure 1.

Typing Derivations ($\mathcal{D}, \mathcal{E}, \mathcal{F}, \ldots$) are written as $\vdash P :: \mathcal{G}$ $^{\mathcal{D}}$ and are read as the derivation $\mathcal{D}$ having the judgement $\vdash P :: \mathcal{G}$ as its conclusion. The meaning of this statement is that the process, P, appearing in the judgement concluding a derivations is well-typed.

Multiplicative Fragment

$$\frac{\vdash P :: \mathcal{G} \mid \Gamma, x : A \mid \Delta, y : A^{\perp}}{\vdash \nu xy\, P :: \mathcal{G} \mid \Gamma, \Delta}\ \text{CUT} \qquad \frac{\vdash P :: \mathcal{G} \quad \vdash Q :: \mathcal{H}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}}\ \text{MIX} \qquad \frac{}{\vdash \mathbf{0} :: \varnothing}\ \text{MIX}_0$$

$$\frac{\vdash P :: \Gamma, y : A \mid \Delta, x : B}{\vdash x[y].P :: \Gamma, \Delta, x : A \otimes B}\ \otimes \qquad \frac{\vdash P :: \varnothing}{\vdash x[].P :: x : 1}\ 1 \qquad \frac{\vdash P :: \Gamma, y : A, x : B}{\vdash x(y).P :: \Gamma, x : A \parr B}\ \parr \qquad \frac{\vdash P :: \Gamma}{\vdash x().P :: \Gamma, x : \bot}\ \bot$$

Additional rules for Additives, Quantifiers and Exponentials

$$\frac{\vdash P :: \Gamma, x : A}{\vdash x[\text{L}].P :: \Gamma, x : A \oplus B}\ \oplus_1 \qquad \frac{\vdash P :: \Gamma, x : B}{\vdash x[\text{R}].P :: \Gamma, x : A \oplus B}\ \oplus_2 \qquad \frac{\vdash P :: \Gamma, x : A \quad \vdash Q :: \Gamma, x : B}{\vdash x.\text{case}\{\text{L} : P, \text{R} : Q\} :: \Gamma, x : A \,\&\, B}\ \&$$

$$\frac{}{\vdash x \leftrightarrow y :: x : A^{\perp}, y : A}\ \text{AX} \qquad \frac{\vdash P :: \Gamma, x : A}{\vdash x[\text{USE}].P :: \Gamma, x : ?A}\ ? \qquad \frac{\vdash P :: \Gamma}{\vdash x[\text{DISP}].P :: \Gamma, x : ?A}\ \text{W} \qquad \frac{\vdash P :: \Gamma, x : ?A, x' : ?A}{\vdash x[\text{DUP}](x').P :: \Gamma, x : ?A}\ \text{C}$$

$$\frac{\vdash P :: z_1 : ?B_1, \ldots, z_n : ?B_n, x : A}{\vdash\, !x\langle z_1, \ldots, z_n\rangle.\{P\} :: z_1 : ?B_1, \ldots, z_n : ?B_n, x : !A}\ ! \qquad \frac{\vdash P :: \Gamma, x : B\{A/X\}}{\vdash x[A].P :: \Gamma, x : \exists X.B}\ \exists \qquad \frac{\vdash P :: \Gamma, x : B \quad X \notin \text{ft}(\Gamma)}{\vdash x(X).P :: \Gamma, x : \forall X.B}\ \forall$$

Fig. 1. πLL, typing rules.

The multiplicative rules form the core of the system. Rule CUT composes two processes in P sharing dual endpoints $x : A$ and $y : A^{\perp}$, hiding them via restriction. This is the logical equivalent of communication. Rule MIX and rule MIX_0 compose independent processes in parallel and introduce the terminated process, respectively.

Rule ⊗ and rule ⅋ type output and input. Note that rule ⊗ types the sent name y in a *separate* environment from the continuation, reflecting that the two are independent resources. By contrast, rule ⅋ keeps y and the continuation x in the *same* environment, reflecting their sequential dependency. The units rule 1 and rule ⊥ type empty output and empty input, where rule 1 requires the continuation to be well-typed in the empty hyperenvironment, meaning P is necessarily semantically equivalent to $\mathbf{0}$.

The additive rules (rule $\oplus_1$, rule $\oplus_2$, and rule &) type selection and offering of alternative processing paths. Note that rule & requires both branches to be typed in the *same* context Γ, reflecting that the choice of branch is made by the other endpoint.

The exponential rules type server replication. The rule ! requires all free names except x to be typed using ?, enforcing that a server only depends on other servers. The rule ?, rule W, and rule C allow a client to use, discard, or duplicate a server respectively.

The rule AX types a link between two dual endpoints, $x \leftrightarrow y$, forwarding all messages sent on x safely to y and vice versa. Rule ∃ and rule ∀ type polymorphic communication, where the side condition $X \notin \mathrm{ft}(\Gamma)$ in rule ∀, ensures the introduced type variable, X, does not accendentally shadow one already existing in Γ.

Example 2.2. The process from Example 2.1 has the typing $\vdash \nu zx\,(z().y[].\mathbf{0} \mid x[].\mathbf{0}) :: y : 1$, as shown by the derivation below:

$$\dfrac{\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\mathrm{MIX}_0}{\vdash x[].\mathbf{0} :: x : 1}\,1 \qquad \dfrac{\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\mathrm{MIX}_0}{\vdash y[].\mathbf{0} :: y : 1}\,1}{\vdash z().y[].\mathbf{0} :: y : 1, z : \bot}\,\bot}{\vdash x[].\mathbf{0} \mid z().y[].\mathbf{0} :: x : 1 \mid y : 1, z : \bot}\,\mathrm{MIX}}{\vdash \nu xz\,(x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: y : 1}\,\mathrm{CUT}$$

Note that, for the typing context $x : 1 \mid y : 1, z : \bot$ to align with the premise of rule CUT, we utilize the monoidal properties of environments and hyperenvironments. In particular, using their identity laws, we instantiate the meta-variables of the rule as $\mathcal{G} = \varnothing$ and $\Gamma = \bullet$. Under these instantiations, the premise of rule CUT: $\vdash P :: \mathcal{G} \mid \Gamma, x : A \mid \Delta, y : A^{\perp}$ can be viewed as $x : A \mid \Delta, y : A^{\perp}$. This effectively instantiates the CUT over the dual types $A = 1$ and $A^{\perp} = \bot$ on channels x and y, with $\Gamma = y : 1$, while ensuring the restricted endpoints implement dual session behaviours, fulfilling the requirements for a creating a valid CUT (session).

2.5 Operational Semantics

The operational semantics of πLL are given as a labelled transition system (LTS) defined over *typing derivations*, in which processes evolve by performing actions on their free names. The semantics follow a Structural Operational Semantics (SOS) style presentation and form the basis for the metatheoretic results of the calculus. We will be focusing on the multiplicative fragment for this part, which is given in Figure 2. The full LTS is given in Montesi and Peressotti [2021].

Actions are defined in the set $Act = \{x[], x(), x[y], x(y) \mid x, y \text{ names}\}$, representing empty output, empty input, name output, and name input respectively. These come together with the silent label τ for internal communication and the parallel composition of labels $l \mid l'$ to form the set of transition labels, defined as $Lbl = \{\tau, l, (l|l') \mid l, l' \in Act, \mathrm{i}(l) \cap \mathrm{i}(l') = \emptyset\}$. The restriction imposed on the parallel composition of labels exists to ensure two labels do not introduce the same name, which would break linear resource management.

$$\frac{}{\vdash P :: \varnothing,\ x:1}\ 1 \quad \xrightarrow{x[]} \quad \vdash P :: \varnothing$$

$$\frac{\vdash P :: \Gamma, x':A \mid \Delta, x:B}{\vdash x[x'].P :: \Gamma, \Delta, x:A \otimes B}\ \otimes \quad \xrightarrow{x[x']} \quad \vdash P :: \Gamma, x':A \mid \Delta, x:B$$

$$\frac{\vdash P :: \Gamma}{\vdash x().P :: \Gamma, x:\bot}\ \bot \quad \xrightarrow{x()} \quad \vdash P :: \Gamma$$

$$\frac{\vdash P :: \Gamma, x':A, x:B}{\vdash x(x').P :: \Gamma, x:A \parr B}\ \parr \quad \xrightarrow{x(x')} \quad \vdash P :: \Gamma, x':A, x:B$$

$$\frac{\vdash P :: \mathcal{G} \xrightarrow{l} \vdash P' :: \mathcal{G}' \qquad \mathrm{i}(l) \cap \mathrm{f}(Q) = \emptyset}{\dfrac{\vdash P :: \mathcal{G} \quad \vdash Q :: \mathcal{H}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}}\ \text{MIX} \xrightarrow{l} \dfrac{\vdash P' :: \mathcal{G}' \quad \vdash Q :: \mathcal{H}}{\vdash P' \mid Q :: \mathcal{G}' \mid \mathcal{H}}\ \text{MIX}}\ \text{PAR}_1$$

$$\frac{\vdash Q :: \mathcal{H} \xrightarrow{l} \vdash Q' :: \mathcal{H}' \qquad \mathrm{i}(l) \cap \mathrm{f}(P) = \emptyset}{\dfrac{\vdash P :: \mathcal{G} \quad \vdash Q :: \mathcal{H}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}}\ \text{MIX} \xrightarrow{l} \dfrac{\vdash P :: \mathcal{G} \quad \vdash Q' :: \mathcal{H}'}{\vdash P \mid Q' :: \mathcal{G} \mid \mathcal{H}'}\ \text{MIX}}\ \text{PAR}_2$$

$$\frac{\vdash P :: \mathcal{G} \xrightarrow{l} \vdash P' :: \mathcal{G}' \qquad \vdash Q :: \mathcal{H} \xrightarrow{l'} \vdash Q' :: \mathcal{H}' \qquad \mathrm{i}(l|l') \cap \mathrm{f}(P \mid Q) = \emptyset}{\dfrac{\vdash P :: \mathcal{G} \quad \vdash Q :: \mathcal{H}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}}\ \text{MIX} \xrightarrow{l|l'} \dfrac{\vdash P' :: \mathcal{G}' \quad \vdash Q' :: \mathcal{H}'}{\vdash P' \mid Q' :: \mathcal{G}' \mid \mathcal{H}'}\ \text{MIX}}\ \text{SYN}$$

$$\frac{P =_\alpha Q \qquad \vdash Q :: \mathcal{G} \xrightarrow{l} \vdash Q' :: \mathcal{G}'}{\vdash P :: \mathcal{G} \xrightarrow{l} \vdash Q' :: \mathcal{G}'}\ =_\alpha \qquad \frac{\dfrac{\vdash P :: \mathcal{G} \mid x:1 \mid \Gamma, y:\bot \quad \xrightarrow{x[]\mid y()} \quad \vdash P' :: \mathcal{G} \mid \Gamma}{\vdash P :: \mathcal{G} \mid x:1 \mid \Gamma, y:\bot}\ 1\bot \quad \xrightarrow{\tau} \quad \vdash P' :: \mathcal{G} \mid \Gamma}{\vdash \nu xy\, P :: \mathcal{G} \mid \Gamma}\ \text{CUT}$$

$$\frac{\vdash P :: \mathcal{G} \mid \Gamma, \Delta, x:A \otimes B \mid \Xi, y:A^\perp \parr B^\perp \quad \xrightarrow{x[x']\mid y(y')} \quad \vdash P' :: \mathcal{G} \mid \Gamma, x:B \mid \Delta, x':A \mid \Xi, y:B^\perp, y':A^\perp}{\dfrac{\vdash P :: \mathcal{G} \mid \Gamma, \Delta, x:A \otimes B \mid \Xi, y:A^\perp \parr B^\perp}{\vdash \nu xy\, P :: \mathcal{G} \mid \Gamma, \Delta, \Xi}\ \text{CUT} \quad \xrightarrow{\tau} \quad \dfrac{\dfrac{\vdash P' :: \mathcal{G} \mid \Gamma, x:B \mid \Delta, x':A \mid \Xi, y:B^\perp, y':A^\perp}{\vdash \nu x'y'\, P' :: \mathcal{G} \mid \Gamma, x:B \mid \Delta, \Xi, y:B^\perp}\ \text{CUT}}{\vdash \nu xy\, \nu x'y'\, P' :: \mathcal{G} \mid \Gamma, \Delta, \Xi}\ \text{CUT}}\ \otimes\parr$$

$$\frac{\vdash P :: \mathcal{G} \mid \Gamma, x:A \mid \Delta, y:A^\perp \xrightarrow{l} \vdash P' :: \mathcal{G}' \mid \Gamma', x:A \mid \Delta', y:A^\perp \qquad x, y \notin \mathrm{f}(l) \cup \mathrm{i}(l)}{\dfrac{\vdash P :: \mathcal{G} \mid \Gamma, x:A \mid \Delta, y:A^\perp}{\vdash \nu xy\, P :: \mathcal{G} \mid \Gamma, \Delta}\ \text{CUT} \xrightarrow{l} \dfrac{\vdash P' :: \mathcal{G}' \mid \Gamma', x:A \mid \Delta', y:A^\perp}{\vdash \nu xy\, P' :: \mathcal{G}' \mid \Gamma', \Delta'}\ \text{CUT}}\ \text{RES}$$

Fig. 2. πMLL, transition rules for derivations.

$$\dfrac{\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0}{\vdash x[].\mathbf{0} :: x : 1}\,1 \qquad \dfrac{\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0}{\vdash y[].\mathbf{0} :: y : 1}\,1}{\vdash z().y[].\mathbf{0} :: y : 1, z : \bot}\,\bot}{\dfrac{\vdash x[].\mathbf{0} \mid z().y[].\mathbf{0} :: x : 1 \mid y : 1, z : \bot}{\vdash \nu xz\,(x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: y : 1}\,\text{CUT}}\,\text{MIX}$$

$\downarrow_\tau$ (by rules SYN and $1\bot$, and the transition axioms for rules 1 and $\bot$)

$$\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0 \qquad \dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0}{\vdash y[].\mathbf{0} :: y : 1}\,1}{\vdash \mathbf{0} \mid y[].\mathbf{0} :: \varnothing \mid y : 1}\,\text{MIX}$$

$\downarrow_{y[]}$ (by rule PAR_2, the transition axiom for rule 1, process congruence, and the monoidal structure of typing environments)

$$\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0 \qquad \dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0}{\vdash \mathbf{0} \mid \mathbf{0} :: \varnothing \mid \varnothing}\,\text{MIX}$$

Fig. 3. An execution for the derivation in Example 2.2.

Each logical rule (1, $\bot$, $\otimes$, $\parr$) has a corresponding transition axiom. These follow the prefix-continuation pattern $\pi.P \xrightarrow{\pi} P$, where performing the action π leads to the remainder of the process. Rule SYN allows for the pairing of concurrent actions via $l|l'$, which is used to facilitate communication by letting two endpoints connected by a restriction perform compatible actions. When two compatible actions are performed over endpoints connected by a restriction, rules $1\bot$ and $\otimes\parr$ simplify the term into an internal τ-transition, modeling the synchronization of the session.

Example 2.3. Figure 3 shows a possible execution for the derivation of the process from Example 2.1.

2.6 Process Semantics and Erasure

While the typing derivations of πLL dictate the valid interactions within a session, the underlying process terms possess a standard, untyped operational behavior that closely mirrors the classic π-calculus. To formally relate the typed derivations to their untyped execution, we follow [Montesi and Peressotti 2021] and define an erasure function, $proc : Der \to Proc$, which strips away the typing information from a derivation. Recursively, $proc$ maps the head of a typing derivation to the specific process constructor it introduces, continuing structurally onto the process's continuation. For any well-typed πLL term, $proc(\mathcal{D})$ effectively extracts the pure process term from the derivation's conclusion.

$$\dfrac{\mathcal{G}}{\vdash \nu xz\,(z().y[].\mathbf{0} \mid x[].\mathbf{0} \mid w().v[].\mathbf{0}) :: w : \bot, v : 1 \mid y : 1}\,\text{CUT}$$

The semantics for process terms must inherently agree with the semantics of derivations for well-typed terms. As formulated in Montesi and Peressotti [Montesi and Peressotti 2021], this relationship is established functorially. Let $\mathcal{P}(X) = \{X' \mid X' \subseteq X\}$ and $\mathcal{L}(X) = X \times \text{Lbl}$ be endofunctors representing the states and labeled transitions, respectively. The erasure function $proc$ acts as a homomorphism from the Labeled Transition System (LTS) of derivations to the LTS of processes, ensuring that the square of these LTS mappings commutes.

$$x[x'].P \xrightarrow{x[x']} P \qquad x(x').P \xrightarrow{x(x')} P \qquad x[].P \xrightarrow{x[]} P \qquad x().P \xrightarrow{x()} P$$

$$\frac{P \xrightarrow{l} P' \quad \mathrm{i}(l) \cap \mathrm{f}(Q) = \emptyset}{P \mid Q \xrightarrow{l} P' \mid Q}\ \mathrm{PAR}_1 \qquad \frac{Q \xrightarrow{l} Q' \quad \mathrm{i}(l) \cap \mathrm{f}(P) = \emptyset}{P \mid Q \xrightarrow{l} P \mid Q'}\ \mathrm{PAR}_2$$

$$\frac{P \xrightarrow{l} P' \quad Q \xrightarrow{l'} Q' \quad \mathrm{i}(l|l') \cap \mathrm{f}(P \mid Q) = \emptyset}{P \mid Q \xrightarrow{l|l'} P' \mid Q'}\ \mathrm{SYN} \qquad \frac{P \xrightarrow{x[x']|y(y')} P'}{\nu xy P \xrightarrow{\tau} \nu xy\, \nu x'y'\, P'}\ \otimes\parr$$

$$\frac{P \xrightarrow{x[]|y()} P'}{\nu xy P \xrightarrow{\tau} P'}\ 1\bot \qquad \frac{P \xrightarrow{l} P' \quad x, y \notin \mathrm{f}(l) \cup \mathrm{i}(l)}{\nu xy P \xrightarrow{l} \nu xy P'}\ \mathrm{RES} \qquad \frac{P =_\alpha Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'}\ =_\alpha$$

$$\mathrm{i}(l|l') = \mathrm{i}(l) \cup \mathrm{i}(l')$$
$$\mathrm{f}(l|l') = \mathrm{f}(l) \cup \mathrm{f}(l')$$
$$\mathrm{i}(\tau) = \emptyset$$
$$\mathrm{f}(\tau) = \emptyset$$
$$\mathrm{i}(x[]) = \mathrm{i}(x()) = \emptyset$$
$$\mathrm{f}(x[]) = \mathrm{f}(x()) = \{x\}$$
$$\mathrm{i}(x[x']) = \mathrm{i}(x(x')) = \{x'\}$$
$$\mathrm{f}(x[x']) = \mathrm{f}(x(x')) = \{x\}$$

Fig. 4. πMLL, transition rules for processes.

In the context of operational semantics, this functorial homomorphism yields a standard Operational Correspondence (or Erasure) theorem. It guarantees that the Labeled Transition System for derivations and the SOS for untyped processes simulate each other exactly:

Theorem 2.4 (Operational Correspondence). *Let $\mathcal{D}$ be a valid typing derivation such that $proc(\mathcal{D}) = P$.*

(1) ***Soundness of Erasure:*** *If $\mathcal{D} \xrightarrow{l} \mathcal{D}'$, then $proc(\mathcal{D}) \xrightarrow{l} proc(\mathcal{D}')$.*

(2) ***Completeness of Erasure:*** *If $proc(\mathcal{D}) \xrightarrow{l} P'$, then there exists a derivation $\mathcal{D}'$ such that $\mathcal{D} \xrightarrow{l} \mathcal{D}'$ and $proc(\mathcal{D}') = P'$.*

This theorem allows us to define the operational target for πLL processes without the syntactic overhead of typing environments. Using the erasure formulation, we can transform the operational semantics for derivations directly into an SOS specification for processes.

Remark 2.5 (On the Redundancy of Side Conditions). The untyped LTS for processes relies heavily on explicit side conditions regarding free and introduced names to prevent the collision of bound variables (e.g., ensuring $i(l) \cap i(l') = \emptyset$). However, when operating at the level of typing derivations, these side conditions become inherently redundant. Because πLL enforces linear resource management, the structural rules of the typed LTS dictate that hyperenvironments can only be merged when their domains are strictly disjoint. Consequently, any well-typed derivation inherently satisfies the name-distinctness and freshness requirements of the underlying untyped calculus, shifting the burden of safety from dynamic transition checks to static structural typing.

2.7 Operational Semantics for Typing Environments

3 πMLL: Paper Level Definitions and Syntax

4 Operational Semantics

Fragment decomposition. The system πLL can be naturally decomposed into two complementary fragments. The first is the multiplicative fragment, πMLL, which includes constructs such as tensor, par, and their associated units. This fragment captures the core structure of interaction, describing how processes communicate and synchronize through linear channels. The second is the non-multiplicative fragment, comprising additives, exponentials, and quantifiers. These constructs enrich the language with additional expressive power, enabling features such as internal and external choice, controlled duplication and disposal of resources, and polymorphic session behavior.

In this development, the syntactic and typing components are defined for the full πLL system, encompassing both fragments. However, the operational semantics and associated metatheory are established for the multiplicative fragment, which suffices to capture the essential communication behavior underlying the system. Extending the operational treatment to the non-multiplicative fragment introduces additional considerations, such as choice resolution and resource management, and is left for future work.

5 Binding and Variable Representation

Formal systems for process calculi and typed π-calculi typically rely on named variables together with a notion of α-equivalence to identify terms that differ only by renaming of bound variables. While conceptually natural, this approach introduces a significant burden in mechanized developments, where reasoning modulo renaming must be made explicit through auxiliary lemmas and infrastructure for capture-avoiding substitution.

In the setting of πLL, this issue is particularly pronounced due to the interplay between linear typing discipline and the structure of communication processes. Bound channel names appear both in process syntax and in typing derivations, and reasoning about their interaction requires repeated handling of freshness conditions and renaming invariance.

Approach. To address this, we adopt a *locally nameless representation* of binders. In this approach, bound variables are represented using de Bruijn indices, while free variables are represented by named identifiers. This hybrid representation eliminates the need for α-equivalence on bound variables, while preserving readability for free variables and top-level structure. Under this representation, α-equivalent terms are identified by construction rather than by quotienting, and variable binding becomes a syntactic mechanism rather than a semantic equivalence relation. This significantly simplifies the treatment of substitution and binding-related lemmas in the mechanization.

Comparison with the standard presentation. In the original formulation of πLL, processes are typically defined using named binders together with an implicit identification of terms up to α-equivalence. While this presentation is intuitive and widely used in the literature, it requires additional metatheoretic work to ensure that all definitions and proofs are invariant under renaming of bound variables. Our locally nameless reformulation avoids this layer of reasoning. In particular, substitution is defined in a capture-avoiding manner only for free variables, while bound variables are handled structurally via indices. This removes the need for repeated renaming arguments throughout the development.

Impact on the formalization. This design choice has several consequences for the mechanization in Lean:

- Substitution lemmas become structurally recursive rather than α-conversion dependent.
- Proofs involving freshness conditions are reduced to simple index reasoning.
- Binding-related invariants are enforced syntactically rather than proven separately.

Overall, this representation does not change the underlying theory of πLL, but provides a more robust foundation for formal reasoning. It enables a cleaner and more direct mechanization of the operational semantics and typing system, without the overhead of explicit α-equivalence reasoning.

6 Future Work

Operational semantics. While the current development provides a complete formalization of syntax, typing, and substitution for all constructs, the operational semantics and associated metatheory have been established for the multiplicative fragment not including the restriction rule. A natural direction for future work is to extend the operational semantics by completely incorporating the

res-rule into the proofs of session fidelity. This would likely require additional reasoning about its behavior when interacting under an application of itself (basically an inversion lemma), and the any permutation its associated hyperenvironment might have, which could require the knowledge that a non-active component in a hyperenvironment is preserved across steps.

Extending the formalization to the full πLL system, would be the ultimate end goal, and would require the addition of a constructors for each of the rules in the non-multiplicative fragment of πLL to the already existing TypingStep inductive. An adaptation of all the lemmas pertaining to the properties of the TypingStep relation would need to be updated to account for these new constructors, and similar reasoning as for the res-rule is expected to be needed for each of the new rules to complete their respective cases in the proof for session fidelity.

Given that the foundational infrastructure for binding and substitution is already in place, this extension is expected to integrate smoothly albeit with the existing development maybe a little tediously when it comes to proving the various multi-thousand line permutations lemmas, which will likely be required. Completing these step would then result in a fully mechanized account of πLL, providing a tool for rigid validity checking of the already sound and existing theory

Structural Congruence. Another matter of interest would be addressing the mechanization of structural congruence. In the current implementation, the strict binary tree structure of the inductive types causes certain well-typed, theoretically reducible processes to become artificially stuck. This primarily manifests through the commutativity and associativity of parallel composition ($P \mid_p Q \equiv Q \mid_p P$ and $P \mid_p (Q \mid_p R) \equiv (P \mid_p Q) \mid_p R$), where the synchronization rule strictly expects interacting processes to be immediate siblings in a specific left-to-right syntactic order.Furthermore, mechanizing other standard congruences—such as Scope Extrusion—introduces unique challenges specific to the chosen syntax representation. By adopting a Locally Nameless (LN) approach, the traditional α-equivalence and fresh-name side conditions of Scope Extrusion are elegantly bypassed. However, they are replaced by the complexities of De Bruijn index arithmetic. Pulling a parallel process Q under a restriction binder (par (cut P) Q ≡ cut (par P (shift 0 2 Q))) requires explicitly shifting the bound indices within Q to account for the increased binder depth. Because the typing hyperenvironments are mechanized using a List-based representation, associativity and commutativity of the contexts must be managed through explicit structural permutation rules.

Formally proving that these De Bruijn index manipulations distribute and commute safely across explicit list permutations and structural merges adds substantial proof overhead.

While replacing binary parallel composition with unordered multisets of processes is a common theoretical remedy to these rigidities, implementing this in a dependent type theory like Lean introduces severe dependent elimination issues. Because multisets are formalized as quotients of lists, indexing inductive typing judgments over them destroys syntax-based matching and inversion, drastically complicating proof search. Consequently, future iterations would be best served by maintaining the rigid binary abstract syntax tree and list-based environments. While the existing permutation rules allow for necessary flexibility in the typing context (background), adding a process congruence rule would make the transition system significantly heavier by collapsing the 1-to-1 mapping between syntax and behavior. Instead, extending the semantics with explicit symmetric structural rules (e.g., syn-symm) allows transitions modulo congruence while maintaining the syntactic tractability and index stability essential for manageable proofs in Lean.

References

Luís Caires and Frank Pfenning. 2010. Session Types as Intuitionistic Linear Propositions. In *CONCUR*. 222–236.

Marco Carbone, Sam Lindley, Fabrizio Montesi, Carsten Schürmann, and Philip Wadler. 2016. Coherence Generalises Duality: A Logical Explanation of Multiparty Session Types. In *27th International Conference on Concurrency Theory,*

CONCUR 2016, August 23-26, 2016, Québec City, Canada (LIPIcs, Vol. 59), Josée Desharnais and Radha Jagadeesan (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 33:1–33:15. doi:10.4230/LIPIcs.CONCUR.2016.33

Roberto Di Cosmo and Dale Miller. 2023. Linear Logic. In *The Stanford Encyclopedia of Philosophy* (fall 2023 ed.), Edward N. Zalta and Uri Nodelman (Eds.). Metaphysics Research Lab, Stanford University. https://plato.stanford.edu/archives/fall2023/entries/logic-linear/

Jean-Yves Girard. 1987. Linear Logic. *Theor. Comput. Sci.* 50 (1987), 1–102. doi:10.1016/0304-3975(87)90045-4

Robin Milner, Joachim Parrow, and David Walker. 1992. A Calculus of Mobile Processes, I. *Inf. Comput.* 100, 1 (1992), 1–40.

Fabrizio Montesi and Marco Peressotti. 2021. Linear Logic, the π-calculus, and their Metatheory: A Recipe for Proofs as Processes. *CoRR* abs/2106.11818 (2021). arXiv:2106.11818 doi:10.48550/arXiv.2106.11818

Philip Wadler. 2014. Propositions as sessions. *Journal of Functional Programming* 24, 2–3 (2014), 384–418. Also: ICFP, pages 273–286, 2012.